

Smart Pedometer - Group I

Introduction

Wearable technology is used to monitor several physiological parameters which may affect the way the human body behaves. These parameters can either be external, such as outdoor humidity and pressure, or internal, such as heart rate and body temperature.

Patients can benefit from wearable technology in an invasive and non-invasive way. The aim of creating such technology is to monitor the body's physiology, help in the diagnosis of diseases at an early stage, aid in rehabilitation, and can also be used for treatment delivery. The wearable technology for this project will be a smart pedometer, which will focus on measuring the number of steps a rehab patient takes, using an accelerometer.

Design Idea

To create a functional wearable medical device, the team decided to develop a smart pedometer that features a sensor that is attached to the leg, for users undergoing low impact rehabilitation after leg injuries or post- leg operations. This device can accurately measure the number of steps taken by the user throughout the day, and therefore monitor their rehabilitation prescribed by their doctors. By monitoring the steps taken, the smart pedometer provides real-time feedback to the user about their activity level, and thus whether they have met their daily goal prescribed previously by their doctors. Giving real-time feedback increases the device's effectiveness in encouraging patients to adhere to their rehabilitation regimen. As the user completes their steps, green lights will appear progressively every time 10% of the prescribed steps have been completed. Once the goal has been completed a sound will go off and the circle will light up with a rainbow.

Feasibility

The decision behind the creation of this device is strongly based on the availability in the market. Although many pedometers exist in devices such as Apple Watches¹ and Fitbits², they tend to be very expensive (around £400 for the apple watch, and up to £130 for the FitBit), making them not accessible to all. Furthermore, the majority of these devices are worn on the arm: they count the steps taken with both legs, and can sometimes confuse hand movement for steps too.³ In fact, problems have risen with Apple watches related to sensors being too dirty, wrist-bands being too loose and not being in contact with the skin, therefore counting steps in an inaccurate way⁴.

For this reason, creating a pedometer which can be strapped on the leg which needs rehabilitation was believed to be feasible. The prototype only counts the steps taken on the injured leg, ensuring that it specifically completes the exercise needed for it to heal. The prototype is adjustable, comfortable and cheaper than many alternatives on the market, making it desirable.

¹ Apple. *A healthy leap ahead. With a healthy dose of style.* Available at: <https://www.apple.com/uk/shop/buy-watch/apple-watch>

² FitBit. *Fitbit at Argos.* Available at: <https://www.argos.co.uk/brands/fitbit>

³ Mashable. Available at: <https://mashable.com/article/fitbit-step-counter/>

⁴ Miles, B. (2022). *Apple Watch Not Tracking Steps? There's a Fix For That.* Available at: <https://www.lifewire.com/fix-apple/>.

Furthermore, it stands out from other pedometers through its LED system, which motivates its users to complete their steps in order to see a rainbow of LEDs and a celebration sound when their goal is reached. This system is more encouraging than a simple black screen displaying digits, and can significantly impact the way a patient views their rehabilitation journey (completing a circle of colors rather than just count numbers).

Need

Due to the existing demand for such devices, the decision to create a smart pedometer to measure the steps taken throughout the day was necessary. One of the main reasons for the current demand of this device is the need for rehabilitation after leg injuries and/or surgeries, where weak muscles⁵ require careful monitoring and exercise to obtain a full recovery. Nevertheless, this device does not only benefit the patients, but can also aid healthcare professionals to remotely monitor the progress of their patients and store their progress data, as well as adjust their rehab treatment plans to obtain a full recovery.

Although this device is primarily designed for post-surgery rehabilitation of leg injuries, it can also be utilized for other conditions that involve monitoring step count.⁶ Thus, the development of this smart pedometer tracker serves as an easily accessible medical help, providing patients with the necessary monitoring and motivation to achieve their daily goals for recovery and overall health.

Technical

The main purpose of this device is to measure the number of steps taken by the patient to therefore know if they have reached the goal prescribed by the healthcare professional as part of their daily rehabilitation. This Smart Pedometer is equipped with a LilyPad Accelerometer ADXL335, which can detect the movement of the injured leg during walking. This data is then processed to provide information about the user's level of activity.

The range of the LilyPad Accelerometer ADXL335 sensor is $\pm 3g$, meaning that it can measure acceleration up to 3 times the force of gravity in any direction. The accuracy of this sensor varies depending on factors such as vibration or temperature. According to the data sheet⁷ of the accelerometer, the typical sensitivity is 330mV/g, and the accuracy is typically $\pm 10\%$ of the measured value.

The accuracy of the accelerometer depends on the application of it. For it to be used for rehab leg injury patient, the $\pm 3g$ range may be enough to measure accurately and precisely the steps taken during low-impact exercise. The average person will take 1 step per second, the pedometer created can take readings every 100 milliseconds. This therefore means it is able to take accurate readings within an accurate time frame. Nonetheless, if the rehab consisted in high-impact activities such as jumping or running, a sensor with a higher range of accuracy may be needed for these applications.

⁵ eOrthopod. Available at: <https://eorthopod.com/faq/why-do-muscles-shrink/>.

⁶ ResearchGate. Available at: <https://www.researchgate.net/publication/330726797/>

⁷ SparkFun. Available at: <https://www.sparkfun.com/datasheets/DevTools/LilyPad/ADXL335.pdf>

	Actual number of steps taken	
Steps measured by Smart pedometer	Step taken(+)	No step taken(-)
Step taken(+)	18	1
No step taken(-)	6	2

The smart pedometer has a sensitivity of 78% and a specificity of 66% in monitoring the number of steps taken by the user (Appendix II). This demonstrated that the smart pedometer was very precise when recording a step taken, however it was slightly less precise when discriminating between an actual step taken and not. Overall, this inferred that the smart pedometer was effective when monitoring the steps taken.

Design of prototype

Circuit Diagram of connections:

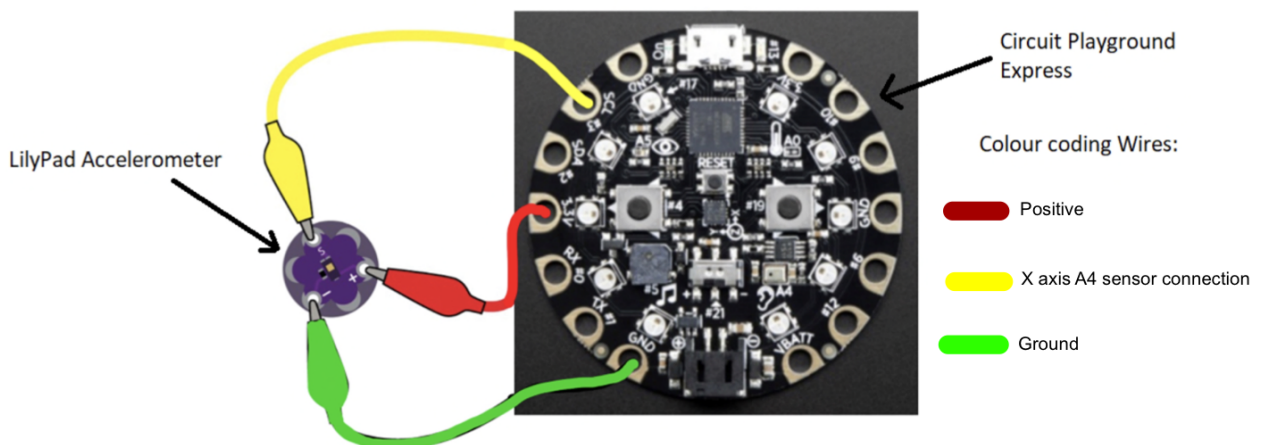


Figure 1: Circuit diagram of the connection between the circuit playground and the Lilipad Accelerometer

List of components:

- 1x Adafruit Circuit Playground
- 1x LilyPad Accelerometer ADXL335
- Silicone glue
- 1x Non slip mat (10cm)
- 2x Luggage straps short adjustable
- 1x Lipo Battery
- 3x Wires
- 1x USB cable

Coding flow chart:

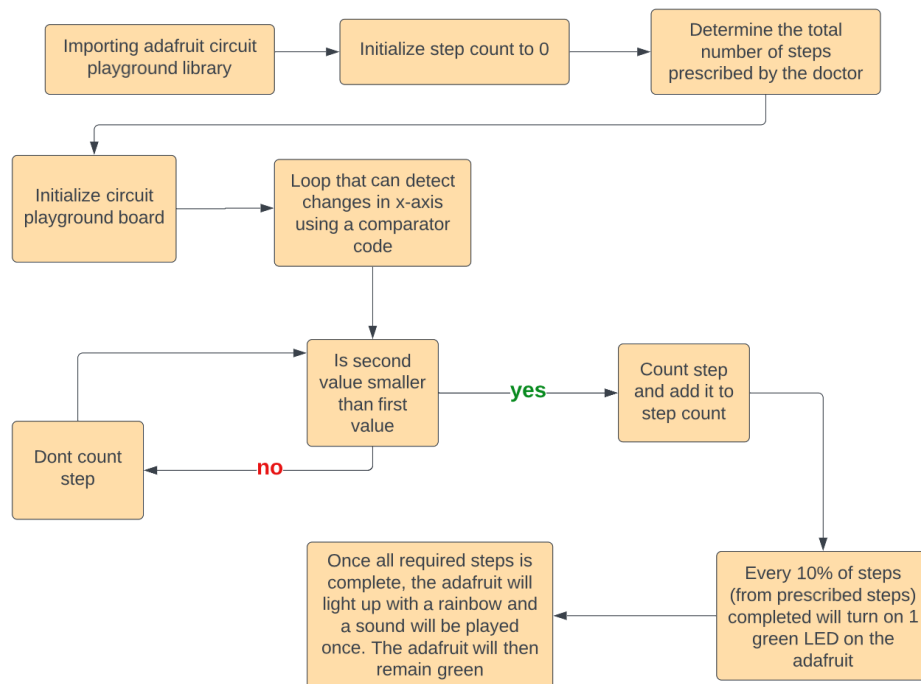


Figure 2: Coding Flowchart used for the coding the smart pedometer prototype

Item Of Clothing:

The item of clothing design can be observed on figure 3.

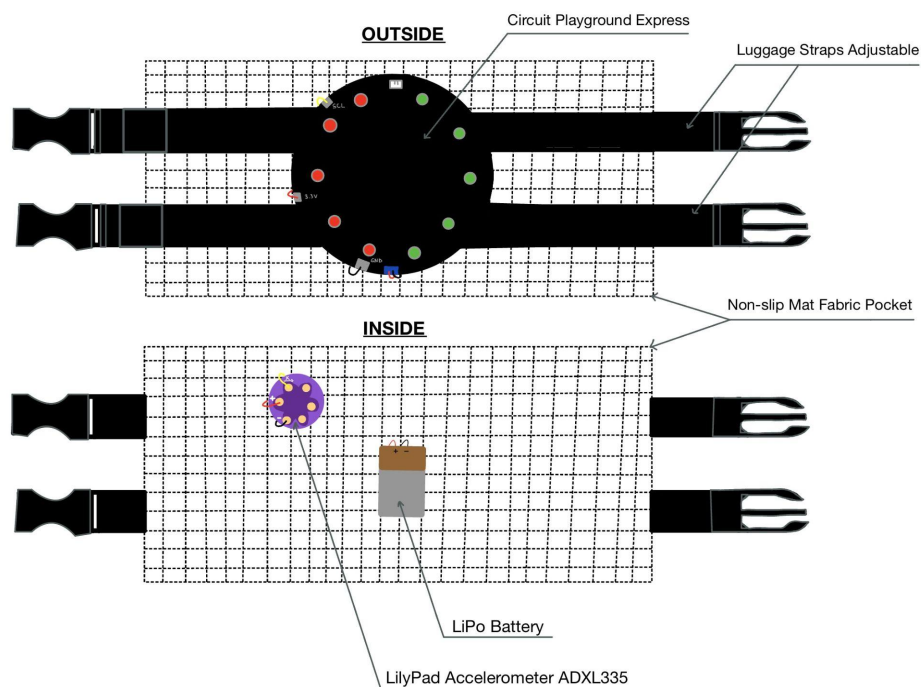


Figure 3: Diagram of the prototype design used for the Smart Pedometer

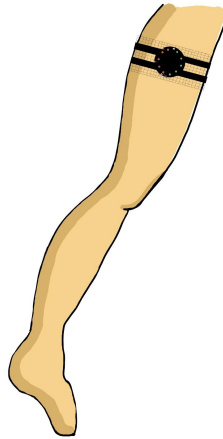


Figure 4: Diagram of the prototype design on a patient's leg.

Bill of materials:

Material	Cost
1x Circuit Playground Express https://thepihut.com	£19.50
1x USB cable	£1.66
1x Lilypad Accelerometer ADXL335 https://shop.pimoroni.com	£ 22.20
1x Lipo Battery	£15
2x Luggage straps adjustable https://www.amazon.co.uk/	£4.85
1x Non Slip Mat (10cm) https://www.amazon.co.uk/	£0.30
3x Wires https://www.hobbytronics.co.uk/breadboard-wire	£1.98
Total	£65.50

Alternative products:

Instead of using regular core wires for the connection between the Circuit Playground express and the accelerometer (Figure 1), multicore wires could be used. This is because multicore wires have higher flexibility than normal wires, and therefore are less prone to snap. Another alternative for the design of this pedometer could be the use of a quad brace with a fitted pocket, instead of the non-slip mat and the luggage adjustable straps.

Testing:

Link will present a video of the Smart pedometer measuring 10 steps:

https://drive.google.com/file/d/11qZxFUQPPy_2O-apxkZdApl8FOBcxw78/view?usp=share_link

Clinical Significance

As mentioned previously, this smart pedometer helps monitor the goals set by the patient's doctor as part of their rehabilitation treatment plan. Moreover, it aids healthcare professionals to remotely monitor the progress of their patients and store their progress data, as well as adjust their rehab treatment plans to obtain a full recovery. Furthermore, this device encourages the patients to pursue the needed daily low impact exercise, as part of their rehabilitation, removing the necessity of tracking the steps on their own and therefore reducing inaccuracy when completing the plan.

The smart pedometer is a valuable tool for monitoring daily rehabilitation low impact activity, however it is not exempt from certain limitations, including false positives and negatives. In a false positive situation the recorded number of steps would be inaccurate leading to an inaccurate final result of the user activity. As a consequence of this, the patient may believe that they have already finished their daily rehabilitation, when in fact they have not. This could lead to a long term negative effect on their treatment plan and thus in their full recovery. In a false negative situation, the opposite would take place: less number of steps would be recorded than the actual completed. This would lead to overtraining the injured leg, which in a long term could end up in a second injury.

Discussion & Conclusion

When designing the smart pedometer, it is important to address the limitations seen. This allows the development and improvement of the algorithms to provide and obtain a more accurate, precise and reliable set of data about the rehabilitation taking place.

Furthermore, other designs including Apple Watches are able to mitigate these potential inaccuracies with the use of artificial intelligence (AI) algorithms.⁸ The AI can accurately detect the "shape" of the step taken by the user. The addition of AI to the smart pedometer created would decrease inaccuracies. Along with the addition of AI it is important to ensure that the device is properly worn on the thigh to prevent slipping, as well as the introduction to the step count of false movements that may confuse the sensor's ability to detect steps.

Another present limitation is the unique use of only the x-axis for step detection. This could be mitigated with the use of the x, y and z-axis to allow for more accurate step detection. Moreover, if this device was used in different terrains, such as uphill and downhill surfaces another limitation would arise, resulting in inaccurate step counts and potentially impact the overall effectiveness of the smart pedometer. This further expresses the need of the y and z-axis sensors. This limitation could also be avoided with the addition of other more accurate sensors that detect changes in elevation and adjust the step count accordingly.

⁸ Apple. Available at: <https://discussions.apple.com/thread/7908894>

The adoption of these measures could significantly reduce the error margin and enhance precision and accuracy and thus reliability of the smart pedometer as a technique to monitor low impact rehabilitation exercises for leg injured patients.

References:

Amazon.co.uk. (n.d.). *Amazon.co.uk*. Available at: <https://www.amazon.co.uk/>

Apple (n.d.) *Apple*. Available at:

https://www.apple.com/uk/shop/buy-watch/apple-watch?option.watch_cases=MNLU3B/A&option.watch_bands=MR2P3ZM/A&preSelect=false&product=Z0YQ&step=detail

Apple. Available at: <https://discussions.apple.com/thread/7908894>

Cool Components. LilyPad Accelerometer ADXL335. Available at:

https://coolcomponents.co.uk/products/lilypad-accelerometer-adxl335?_pos=1&_sid=980288df7&_ss=r

eOrthopod. (n.d.). *Why Do Muscles Shrink?* Available at:

<https://eorthopod.com/faq/why-do-muscles-shrink/>

Fitbits (n.d.). *Fitbits*. Available at :

<https://www.fitbit.com/global/uk/products/smartwatches/sense2?sku=521BKGB>

FitBit. *Fitbit at Argos*. Available at: <https://www.argos.co.uk/brands/fitbit>

HobbyTronics. (n.d.). *Breadboard Wire*. Available at:

<https://www.hobbytronics.co.uk/breadboard-wire>

Mashable. Available at:

<https://mashable.com/article/fitbit-step-counter-accuracy#:~:text=Make%20sure%20you%20are%20wearing%20your%20Fitbit%20device%20correctly&text=If%20you're%20wearing%20it,a%20shirt%20or%20something%20similar.>

Miles, B. (2022). *Apple Watch Not Tracking Steps? There's a Fix For That*. Available at:

<https://www.lifewire.com/fix-apple-watch-not-tracking-steps-4689002#:~:text=Typical%20causes%20of%20an%20Apple,is%20running%20in%20the%20background.>

Pimoroni. (n.d.). *Pimoroni*. Available at: <https://shop.pimoroni.com>

ResearchGate. (n.d.). *ADXL335 accelerometer datasheet*. Available at:

<https://www.researchgate.net/publication/330726797/>

SparkFun. (n.d.). *ADXL335*. Available at:

<https://www.sparkfun.com/datasheets/DevTools/LilyPad/ADXL335.pdf>

The Pihut. (n.d.). *The Pihut*. Available at: <https://thepihut.com>

Appendix I: Code used for the Smart Pedometer

```
#include <Adafruit_CircuitPlayground.h> //installs the adafruit circuit playground
library

int nb_steps = 0; //initializes an integer value which counts the number of steps
taken
int step_2 = 0; //initializes the value which will hold the previous value taken
from analogRead(A4) and keeps track of it
float total_steps = 10; //Initializes a float value which represents the number of
steps the patient must take (assigned by the doctor).

// It's a float value because the lights will turn on
depending on the percentage of total steps that have been completed.

void setup() {
  Serial.begin(9600); // Initializes the serial communication at a rate of 9600 bits
per second
  CircuitPlayground.begin(); // Initializes the circuit playground board
}

void loop(){
  int step_1 = analogRead(A4); // Initializes a variable which reads the analog
signal obtained at pin A4
  if (step_1 >> step_2) { // Compares the step taken with the step before. If the
current analog value is higher than the one before, then no step will be counted.
    // If however, the current analog step value is lower than
the previous one, then the peak on the plot has passed and a step can therefore be
counted
    nb_steps++; // Increases the number steps (increments it by 1)
  }
  step_2 = step_1; // The new value of step_2 is now step_1. This is to make a new
comparison.
  Serial.print(nb_steps); //Displays the number of steps taken on the serial monitor
(this was used during the testing phase to know if the device was measuring the
steps accurately)
  delay(100); //Delay of 100 ms between each measurement taken

  CircuitPlayground.clearPixels(); //No LEDs are turned on in the beginning.

  if ((nb_steps >= 0) && (nb_steps < 0.1*total_steps)) { //If the number of steps is
under 10% of the total steps needed, all lights are red
    CircuitPlayground.setPixelColor(0, 255, 0, 0); //Turns LED 0 red
    CircuitPlayground.setPixelColor(1, 255, 0, 0); //Turns LED 1 red
    CircuitPlayground.setPixelColor(2, 255, 0, 0); //Turns LED 2 red
    CircuitPlayground.setPixelColor(3, 255, 0, 0); // ...
    CircuitPlayground.setPixelColor(4, 255, 0, 0);
```

```

    CircuitPlayground.setPixelColor(5, 255, 0, 0);
    CircuitPlayground.setPixelColor(6, 255, 0, 0);
    CircuitPlayground.setPixelColor(7, 255, 0, 0);
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}
else if ((nb_steps >= (0.1*total_steps)) && (nb_steps < 0.2*total_steps)) { //if
the number of steps is more than 10% but under 20% of the total steps needed, one
light turns green)
    CircuitPlayground.setPixelColor(0, 0, 255, 0); //Turns LED 0 green
    CircuitPlayground.setPixelColor(1, 255, 0, 0); //Turns LED 1 red
    CircuitPlayground.setPixelColor(2, 255, 0, 0); //Turns LED 2 red
    CircuitPlayground.setPixelColor(3, 255, 0, 0); // ...
    CircuitPlayground.setPixelColor(4, 255, 0, 0);
    CircuitPlayground.setPixelColor(5, 255, 0, 0);
    CircuitPlayground.setPixelColor(6, 255, 0, 0);
    CircuitPlayground.setPixelColor(7, 255, 0, 0);
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}
else if ((nb_steps >= (0.2*total_steps)) && (nb_steps < 0.3*total_steps)) { //Over
20% and under 30% of total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0); //Turns LED 0 green
    CircuitPlayground.setPixelColor(1, 0, 255, 0); //Turns LED 1 green
    CircuitPlayground.setPixelColor(2, 255, 0, 0); //Turns LED 2 red
    CircuitPlayground.setPixelColor(3, 255, 0, 0); //Turns LED 3 red
    CircuitPlayground.setPixelColor(4, 255, 0, 0); // ...
    CircuitPlayground.setPixelColor(5, 255, 0, 0);
    CircuitPlayground.setPixelColor(6, 255, 0, 0);
    CircuitPlayground.setPixelColor(7, 255, 0, 0);
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}
else if ((nb_steps >= (0.3*total_steps)) && (nb_steps < 0.4*total_steps)) { //Over
30% and under 40% of total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0); // 3 LEDs are green
    CircuitPlayground.setPixelColor(3, 255, 0, 0);
    CircuitPlayground.setPixelColor(4, 255, 0, 0);
    CircuitPlayground.setPixelColor(5, 255, 0, 0);
    CircuitPlayground.setPixelColor(6, 255, 0, 0);
    CircuitPlayground.setPixelColor(7, 255, 0, 0);
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}

```

```

}

else if ((nb_steps >= (0.4*total_steps)) && (nb_steps < 0.5*total_steps)) {
//Over 40% and under 50% of total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);
    CircuitPlayground.setPixelColor(3, 0, 255, 0); // 4 LEDs are green
    CircuitPlayground.setPixelColor(4, 255, 0, 0);
    CircuitPlayground.setPixelColor(5, 255, 0, 0);
    CircuitPlayground.setPixelColor(6, 255, 0, 0);
    CircuitPlayground.setPixelColor(7, 255, 0, 0);
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}

else if ((nb_steps >= (0.5*total_steps)) && (nb_steps < 0.6*total_steps)) { //Over
50% and under 60% of total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);
    CircuitPlayground.setPixelColor(3, 0, 255, 0);
    CircuitPlayground.setPixelColor(4, 0, 255, 0); // 5 LEDs are green
    CircuitPlayground.setPixelColor(5, 255, 0, 0);
    CircuitPlayground.setPixelColor(6, 255, 0, 0);
    CircuitPlayground.setPixelColor(7, 255, 0, 0);
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}

else if ((nb_steps >= (0.6*total_steps)) && (nb_steps < 0.7*total_steps)) { //Over
60% and under 70% of total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);
    CircuitPlayground.setPixelColor(3, 0, 255, 0);
    CircuitPlayground.setPixelColor(4, 0, 255, 0);
    CircuitPlayground.setPixelColor(5, 0, 255, 0); // 6 LEDs are green
    CircuitPlayground.setPixelColor(6, 255, 0, 0);
    CircuitPlayground.setPixelColor(7, 255, 0, 0);
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}

else if ((nb_steps >= (0.7*total_steps)) && (nb_steps < 0.8*total_steps)) { //Over
70% and under 80% of total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);

```

```

CircuitPlayground.setPixelColor(3, 0, 255, 0);
CircuitPlayground.setPixelColor(4, 0, 255, 0);
    CircuitPlayground.setPixelColor(5, 0, 255, 0);
CircuitPlayground.setPixelColor(6, 0, 255, 0); //7 LEDs are green
CircuitPlayground.setPixelColor(7, 255, 0, 0);
CircuitPlayground.setPixelColor(8, 255, 0, 0);
CircuitPlayground.setPixelColor(9, 255, 0, 0);
}

else if ((nb_steps >= (0.8*total_steps)) && (nb_steps < 0.9*total_steps)) { //Over
80% and under 90% of total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);
    CircuitPlayground.setPixelColor(3, 0, 255, 0);
    CircuitPlayground.setPixelColor(4, 0, 255, 0);
    CircuitPlayground.setPixelColor(5, 0, 255, 0);
    CircuitPlayground.setPixelColor(6, 0, 255, 0);
    CircuitPlayground.setPixelColor(7, 0, 255, 0); //8 LEDs are green
    CircuitPlayground.setPixelColor(8, 255, 0, 0);
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}

else if ((nb_steps >= (0.9*total_steps)) && (nb_steps < total_steps)) { //Over 90%
and under total steps
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);
    CircuitPlayground.setPixelColor(3, 0, 255, 0);
    CircuitPlayground.setPixelColor(4, 0, 255, 0);
    CircuitPlayground.setPixelColor(5, 0, 255, 0);
    CircuitPlayground.setPixelColor(6, 0, 255, 0);
    CircuitPlayground.setPixelColor(7, 0, 255, 0);
    CircuitPlayground.setPixelColor(8, 0, 255, 0); //9 LEDs are green
    CircuitPlayground.setPixelColor(9, 255, 0, 0);
}

else if (nb_steps == total_steps) { //total steps completed
    CircuitPlayground.setPixelColor(0, 255, 0, 0);
    CircuitPlayground.setPixelColor(1, 128, 128, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);
    CircuitPlayground.setPixelColor(3, 0, 128, 128);
    CircuitPlayground.setPixelColor(4, 0, 0, 255);

    CircuitPlayground.setPixelColor(5, 0xFF0000);
    CircuitPlayground.setPixelColor(6, 0x808000);
    CircuitPlayground.setPixelColor(7, 0x00FF00);
    CircuitPlayground.setPixelColor(8, 0x008080);

```

```

    CircuitPlayground.setPixelColor(9, 0x0000FF);
    //If total steps or more are completed, make a sound
    CircuitPlayground.playTone(500, 100); //plays a sound of 500 Hz for 100 ms
    CircuitPlayground.playTone(700, 100); //plays a sound of 700 Hz for 100 ms
    CircuitPlayground.playTone(900, 100); //plays a sound of 900 Hz for 100 ms
}
else if (nb_steps > total_steps) { // more than total steps completed
    CircuitPlayground.setPixelColor(0, 0, 255, 0);
    CircuitPlayground.setPixelColor(1, 0, 255, 0);
    CircuitPlayground.setPixelColor(2, 0, 255, 0);
    CircuitPlayground.setPixelColor(3, 0, 255, 0);
    CircuitPlayground.setPixelColor(4, 0, 255, 0);
    CircuitPlayground.setPixelColor(5, 0, 255, 0);
    CircuitPlayground.setPixelColor(6, 0, 255, 0);
    CircuitPlayground.setPixelColor(7, 0, 255, 0);
    CircuitPlayground.setPixelColor(8, 0, 255, 0);
    CircuitPlayground.setPixelColor(9, 0, 255, 0); //ALL LEDs are green
}
}

```

Appendix II: Sensitivity & Specificity

Calculations made for the sensitivity:

$$\frac{18}{23} \times 100 = 78\%$$

Calculations made for the specificity:

$$\frac{2}{3} \times 100 = 66\%$$